

JiT: 大 patch Pixel DiT 的训练与理论理解

申家芮

组会：论文讲解

2025 年 12 月 4 日

目录

- 1 前置知识：扩散模型与空间选择
- 2 论文结构与主线
- 3 论文核心内容：方法与实验
- 4 理论解释：流形假设与高维 token
- 5 更深入的对比与个人分析
- 6 相关工作与对比
- 7 总结与展望

① 前置知识：扩散模型与空间选择

② 论文结构与主线

③ 论文核心内容：方法与实验

④ 理论解释：流形假设与高维 token

⑤ 更深入的对比与个人分析

⑥ 相关工作与对比

⑦ 总结与展望

扩散模型发展脉络（1）：DDPM

- DDPM: Denoising Diffusion Probabilistic Models
- 思路：
 - 正向过程：一步步往图像里加高斯噪声。
 - 反向过程：训练一个网络一步步去噪。
- 训练目标：
 - 通常让网络预测加上的噪声 ϵ 。
 - 用 MSE loss: $\|\epsilon_\theta - \epsilon\|^2$ 。
- 问题：
 - 在像素空间上做，维度巨大。
 - 训练和采样步骤都很多，整体非常慢。

扩散模型发展脉络（2）：ADM

- ADM: Diffusion Models Beat GANs on Image Synthesis
- 主要改进：
 - 使用更强的 U-Net 结构。
 - 引入 classifier guidance 提升有条件生成质量。
- 仍然在像素空间：
 - 例如 $256 \times 256 \times 3$ 的图像。
- 代价：
 - 以 ImageNet-256 为例，训练一个好模型需要巨量算力。
 - 扩散的“慢”的印象基本来自这个阶段。

扩散模型发展脉络（3）：为什么要 latent？

- 像素空间的问题：
 - 分辨率高 → 维度高。
 - 每一步前向/反向传播的代价都非常大。
- 直观想法：
 - 能不能先把图像压缩到一个“更小的空间”，再在这个空间扩散？
- 于是引入：
 - VAE, VQ-VAE, VQGAN 等“压缩网络”。
 - Latent Diffusion Model (LDM) 把这些整合进扩散框架。

VAE 基础 (1): 从自编码器到 VAE

- 普通自编码器 (Autoencoder):
 - encoder: $z = f_{\phi}(x)$ (压缩)。
 - decoder: $\hat{x} = g_{\theta}(z)$ (重建)。
 - loss: 重建误差 (例如像素 L2)。
- 问题:
 - z 只是“某种编码”，没有明确的概率分布。
 - 想直接从 z 采样生成新图像很难。
- VAE (Variational Autoencoder):
 - 给 latent z 加上明确的先验分布 $p(z)$ (常用标准高斯)。
 - 同时学习 encoder $q_{\phi}(z|x)$ 和 decoder $p_{\theta}(x|z)$ 。

VAE 基础 (2): ELBO 与训练目标

- VAE 的目标：最大化 $\log p_\theta(x)$ 的下界 (ELBO)：

$$\log p_\theta(x) \geq \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - \text{KL}(q_\phi(z|x) \parallel p(z))$$

- 两部分含义：
 - 重构项：让 decoder 能从 z 重建出 x 。
 - KL 项：让 $q_\phi(z|x)$ 接近先验 $p(z)$ 。
- 实现：
 - encoder 输出 $\mu_\phi(x), \sigma_\phi(x)$ 。
 - 重参数化： $z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon$ 。

VAE 基础 (3): VAE 的“信息瓶颈”

- VAE latent 的大小（通道数、空间分辨率）必须折中：
 - 太小 → 重构模糊，细节丢失。
 - 太大 → 后续在 latent 上做扩散又变得很贵。
- KL 正则：
 - 把 $q(z|x)$ 拉向标准高斯。
 - 有利于采样，但会压缩信息。
- 对 LDM 来说：
 - latent 是一个“必要但受限”的瓶颈。
 - 是效率的来源，也是上限的约束。

VAE 与 LDM 的两阶段训练痛点

- LDM 的做法：
 - 第一步：单独训练 VAE，优化“重构质量”。
 - 第二步：在 VAE latent 上训练扩散模型。
- 潜在问题：
 - VAE 的目标是“还原输入图像”。
 - 扩散的目标是“在 latent 中方便建模分布”。
 - 两者不一定完全一致，latent 空间可能对扩散不够“友好”。
- 这也是为什么很多人对“端到端方案”很感兴趣：
 - 希望避免两阶段目标不一致的问题。

latent-space vs pixel-space: 空间选择

- Pixel-space:
 - 直接在像素上加噪、去噪。
 - 分辨率高时维度巨大，扩散过程很贵。
- Latent-space (LDM):
 - 先用 VAE 压缩到低维 latent。
 - 在 latent 上做扩散，维度更小，训练更快。
- 权衡：
 - pixel-space: 更直接、端到端，但难训。
 - latent-space: 更高效，但引入 VAE 瓶颈和两阶段问题。

Transformer 基础 (1): token 序列视角

- Transformer 的输入是一个 token 序列：
 - 每个 token 是一个向量，维度通常为 hidden size。
- 对于图像：
 - 通常通过 patchify 把图像块当作 token。
- 核心结构：
 - Self-attention: token 与 token 之间的信息交流。
 - MLP: 对每个 token 单独做非线性变换。

Transformer 基础 (2)：注意力的复杂度

- Self-attention 的主要计算：
 - 对每一层： $Q = XW_Q$, $K = XW_K$, $V = XW_V$ 。
 - 注意力： $\text{softmax}(QK^\top / \sqrt{d_k})V$ 。
- 复杂度：
 - 序列长度 N , hidden 维度 d 。
 - $QK^\top$ 是 $N \times N$ 矩阵。
 - 总复杂度约为 $O(N^2 \times d)$ 。
- 启发：
 - 想减轻计算，首先要减小 N (token 数)。
 - patchify 是一种减少 N 的方式。

patchify (1): 基本定义

- 对一张 $H \times W$ 的图像：
 - 选择 patch size 为 p 。
 - 每个 patch 覆盖 $p \times p$ 像素。
- patchify 的过程：
 - 把每个 $p \times p \times 3$ 的小块展平为一个向量。
 - 这个向量就是一个 token（再经过线性层到 hidden）。
- token 数量：

$$N = \frac{H}{p} \times \frac{W}{p}$$

- 每个 token 原始维度：

$$d_{\text{patch}} = p^2 \times 3$$

patchify (2): p 变大会发生什么?

- 以 256×256 的 RGB 图像为例：
 - $p = 2$: $N = 128 \times 128 = 16,384$, $d_{\text{patch}} = 12$ 。
 - $p = 8$: $N = 32 \times 32 = 1,024$, $d_{\text{patch}} = 192$ 。
 - $p = 16$: $N = 16 \times 16 = 256$, $d_{\text{patch}} = 768$ 。
- 直观变化：
 - p 变大 $\rightarrow$ token 数变少 $\rightarrow$ attention 计算变便宜。
 - 但每个 token 维度迅速增大 $\rightarrow$ token 变得“很复杂”。
- 对模型来说：
 - 它要在高维空间中学每个 token 的分布。
 - 这就是后面“大 patch $\times$ 高维 token”的根源。

为什么不能直接不用 patchify?

- 设想一：每个像素一个 token ($p = 1$)
 - 256×256 图像： $N = 65,536$ ，每个 token 维度 $d \approx 3$ 。
 - self-attention 复杂度 $\propto N^2 d \approx (6.5 \times 10^4)^2 \times 3$ ，几乎不可训练。
 - 注意力矩阵大小是 $65,536 \times 65,536$ ，显存直接爆炸。
- 设想二：整图展平成一个 token
 - $N = 1$ ， $d = 256 \times 256 \times 3 \approx 2 \times 10^5$ 。
 - 没有 token 间的 self-attention，网络退化成“大号 MLP”。
 - 参数量巨大，空间结构难以编码。
- 小结：
 - patchify 是在“N 太大”和“没有序列结构”之间的折中。
 - 真正贵的是 N^2 ，不是像素总个数 $H \times W \times 3$ 。

patchify (3): 和 Transformer hidden 的关系

- Transformer 通常有一个固定的 hidden 维度：
 - 例如 768、1024、1536 等。
- patchify 后的处理：
 - 先用线性/卷积层将 d_{patch} 映射到 hidden。
 - 再在 hidden 空间里进行多层 Transformer。
- 一旦 $d_{\text{patch}} > \text{hidden}$ ：
 - 一开始就是“高维 $\rightarrow$ 低维”的强制压缩。
 - 这对学习高维噪声分布来说是极大的挑战。

预测目标扫盲： $x / \epsilon / v$ (1)

- 在 rectified flow 或类似设定下：

$$z_t = tx + (1 - t)\epsilon$$

- x ：干净图像。
 - ϵ ：标准高斯噪声。
 - z_t ：时刻 t 的“带噪图像”。
- 三种常见预测目标：
 - 预测 ϵ ：噪声预测。
 - 预测 x ：干净图像预测。
 - 预测“速度” v ：二者之间的线性组合。

预测目标扫盲： $x / \epsilon / v$ (2)

- 三者之间可以互相转换：
 - 给定任意两个（例如 x 和 z_t ），就能算出另外一个（例如 ϵ 或 v ）。
- 从“表达能力”角度看：
 - 神经网络只要学会其中一个，其他都可以通过公式推回去。
 - 数学上是等价的。
- 但从“好不好学”角度看：
 - 在高维空间中，预测噪声和预测图像难度是不同的。
 - 这正是 JiT 要研究的问题。

① 前置知识：扩散模型与空间选择

② 论文结构与主线

③ 论文核心内容：方法与实验

④ 理论解释：流形假设与高维 token

⑤ 更深入的对比与个人分析

⑥ 相关工作与对比

⑦ 总结与展望

JiT 的一句话概括

- 场景：像素空间 (pixel space) + 大 patch 的 Transformer (DiT)。
- 问题：大 patch 时，传统的噪声/速度预测训练极不稳定。
- 方法：在 rectified flow 框架下
 - 将预测目标改为干净图像 x (x-prediction)。
 - 使用基于速度 v 的损失 (v -loss)。
- 结论：在“大 patch + 高维 token + hidden 瓶颈”的 regime 下，

x -prediction 明显好于 v / ϵ -prediction。

论文整体结构（按原文）

- 1. 现象：大 patch pixel DiT 难以训练，质量严重退化。
- 2. 分析：高维 patch token + 噪声/速度预测 → 学习困难。
- 3. 方法：提出 JiT，使用 x-pred + v-loss。
- 4. 结构改动：现代 DiT 细节（bottleneck, RoPE, qk-norm 等）。
- 5. 实验：在不同分辨率和 patch size 上系统验证。
- 6. 理论：用流形假设 + 信息瓶颈解释现象。
- 7. 结论与讨论：定位 JiT 的适用场景与局限。

- 1 前置知识：扩散模型与空间选择
- 2 论文结构与主线
- 3 论文核心内容：方法与实验
- 4 理论解释：流形假设与高维 token
- 5 更深入的对比与个人分析
- 6 相关工作与对比
- 7 总结与展望

Pixel DiT 为什么难？(1)

- 目标：在像素空间，用 ViT/DiT 直接做扩散。
- 高分辨率 + Transformer：
 - 不用 latent，而是直接在 pixel space。
 - 为了减小 token 数 $N \rightarrow$ 不得不增大 patch size。
- 结果：
 - patch token 的维度 d_{patch} 变得非常大。
 - hidden size 通常固定在 768~1800 左右。
 - 出现“高维 token $\rightarrow$ 低维 hidden”的强压缩。

Pixel DiT 为什么难？(2)

- 以前的做法（例如原始 DiT）：
 - 在 latent 空间中使用小 patch ($p = 2$)。
 - patch dim 很小，hidden 足够大。
- 当移到 pixel space：
 - 为了高分辨率，又想保住算力 → 被迫用大 patch ($p = 8, 16$)。
 - patch dim $\gg$ hidden，这会成为模型的主要瓶颈。
- 如果还用 v-pred / ϵ -pred：
 - 目标是高维随机噪声。
 - 对一个 under-complete 的网络来说，几乎学不动。

实验现象：大 patch 下的退化

- 论文中的观察：
 - 小 patch（例如 $p=2,4$ ）下，v-pred 正常工作。
 - patch 变大后（ $p=8$ 、 $p=16$ ），生成质量急剧下降。
- 具体表现：
 - FID 明显升高。
 - 样本质量肉眼可见变差，甚至发散。
- 说明：
 - 固定结构和 hidden，单纯增大 patch 会让 v-pred / ϵ -pred 失效。

JiT 的训练规模与“效率”定位

- 训练配置（论文 + 官方代码）：
 - ImageNet-256/512 上训练 200~600 epoch。
 - 有效 batch 大约在 1024 左右。
 - 每种模型都要完整跑一轮大规模训练。
- 相对 latent DiT / LDM：
 - latent 分辨率更小（例如 32×32 ）。
 - 训练和采样都比 pixel-space 更快。
 - JiT 没有“比 latent-DiT 更快”的优势。
- 相对其他 pixel-space 扩散（如 PixelFlow、部分 SiD2 配置）：
 - JiT 的 token 数较少（例如 256 个 token）。
 - 结构非常简洁，单步 GFLOPs 反而更低。
- 定位：
 - JiT 主要贡献在“可训练性 + 理论解释”，
 - 而不是“全面超越 latent-DiT 的效率与 FID”。

JiT 方法核心 (1): rectified flow 加噪

- 加噪方式:

$$z_t = tx + (1 - t)\epsilon, \quad t \in [0, 1]$$

- 含义:

- $t = 0$ 时, z_t 接近纯噪声。
- $t = 1$ 时, z_t 回到干净图像 x 。

- 可以理解为:

- 沿着“从噪声到图像”的直线移动。
- 模型要学的是这条线上的“速度”。

JiT 方法核心 (2): x-prediction

- 传统做法：
 - 网络直接预测噪声 ϵ_θ 或速度 v_θ 。
- JiT 的做法：
 - 网络直接输出图像预测 $x_\theta(z_t, t)$ 。
 - 即：网络的输出与 x 在同一空间。
- 这样做的直觉：
 - 图像 x 生活在一个低维流形上。
 - 学习图像比学习高维噪声更稳定。

JiT 方法核心 (3): v-loss 的定义

- 定义速度 (flow):

$$v = \frac{x - z_t}{1 - t}$$

- 预测速度:

$$v_\theta = \frac{x_\theta - z_t}{1 - t}$$

- 损失函数:

$$L = \|v_\theta - v\|_2^2$$

- 换一种写法:

$$v_\theta - v = \frac{x_\theta - x}{1 - t} \Rightarrow L = \left\| \frac{x_\theta - x}{1 - t} \right\|^2$$

实际上是对 $(x_\theta - x)$ 在不同 t 上加了权重。

v-loss 的直观理解

- $(1 - t)$ 越小：
 - 图片越接近干净图像。
 - 对误差的权重越大。
- $(1 - t)$ 越大：
 - 图片越接近噪声。
 - 对误差的权重越小。
- 直觉：
 - 更在意后期的误差（接近生成结果）。
 - 对特别 noisy 的阶段要求相对宽松。
- 实验上：
 - $x\text{-pred} + v\text{-loss}$ 比 $x\text{-pred} + x\text{-loss}$ 更稳定。

数值稳定：为什么要 $\text{clamp}(1 - t)$ ？

- 损失中有 $\frac{1}{1-t}$ ：
 - $t \rightarrow 1$ 时， $1 - t \rightarrow 0$ ，可能指数级放大。
- 实际训练中：
 - 如果直接用 $\frac{1}{1-t}$ ，会导致梯度爆炸。
 - 容易出现 loss 为 NaN 的情况。
- JiT 的实现：

$$\frac{1}{1-t} \rightarrow \frac{1}{\max(1-t, 5 \times 10^{-2})}$$

- 理解：
 - 类似对 loss 权重做了“上限裁剪”。
 - 轻微破坏了理论上的精确性，但换来数值稳定。

t 采样：logit-normal 的作用

- 训练时需要从 $[0, 1]$ 中采样 t 。
- 简单做法：均匀采样 $t \sim \mathcal{U}(0, 1)$ 。
- JiT 和很多现代工作（例如 SD3）：
 - 使用 logit-normal 分布采样 t 。
 - 可以控制在哪些“阶段”采样更密集。
- 直觉：
 - 有些区间对图像质量更关键。
 - 更频繁地训练这些关键阶段，有助于整体质量。

结构改动 (1): BottleneckPatchEmbed (概念版)

- JiT 中的 patch embedding 使用了两层卷积：
 - proj1: Conv2d(in=3, out=bottleneck_dim, kernel=patch_size, stride=patch_size)。
 - proj2: Conv2d(in=bottleneck_dim, out=hidden_size, kernel=1, stride=1)。
- proj1 的作用：
 - 每个 $p \times p$ patch 一次性映射到 bottleneck_dim (例如 128/256)。
 - 相当于对 patch 内的 $p^2 \times 3$ 维像素做一个“可学习 PCA”。
- proj2 的作用：
 - 只在通道维上线性变换: bottleneck_dim $\rightarrow$ hidden_size (例如 768/1024)。
 - 在 bottleneck 子空间上“换一组坐标”，同时把特征展开得更宽，方便后续注意力和 MLP 使用。

结构改动（1-续）：BottleneckPatchEmbed 的维度流向

- 以 256×256 , patch=16, JiT-B/16 为例：
 - 输入： $(B, 3, 256, 256)$ 。
 - proj1: $(B, 3, 256, 256) \rightarrow (B, 128, 16, 16)$ 。
 - proj2: $(B, 128, 16, 16) \rightarrow (B, 768, 16, 16)$ 。
 - flatten+transpose: $(B, 768, 16, 16) \rightarrow (B, 256, 768)$ 。
- 其中：
 - 只有 proj1 和 proj2 有参数。
 - flatten(2).transpose(1,2) 只是 reshape, 把 16×16 个 patch 展成 patch 序列。
- 信息瓶颈：
 - bottleneck_dim (128/256) 决定了单个 patch 最多保留多少自由度。
 - hidden_size (768/1024/1280) 只是“展开用的宽度”，不会增加信息维度。

结构改动（2）：Modern DiT 配方

- JiT 借鉴了 LightningDiT 等工作的经验：
 - 使用 SwiGLU 代替 ReLU/GeLU，提升稳定性。
 - 使用 RMSNorm 替代 LayerNorm。
 - 引入 RoPE（旋转位置编码）以更好处理长序列。
 - 在注意力中使用 qk-norm 控制数值范围。
- 整体效果：
 - 提升训练稳定性和最终 FID。
 - 但这些都是结构/工程层面的小加成。

结构改动 (3): class condition 设计

- 针对有类别标签的条件生成：
 - 不再只用一个 class token。
 - 把一个 class token 扩展为一组（例如 32 个）class token。
- 加上 CFG (Classifier-Free Guidance) 相关技巧：
 - 例如 CFG interval, 在部分步数使用 guidance。
- 这些设计：
 - 提升条件生成的表达能力。
 - 与 JiT 的“预测目标改动”是正交的。

实验：不同 patch 下的表现（概览）

- JiT 对比了多种组合：
 - 预测目标： $x / \epsilon / v$ 。
 - 损失空间： $x\text{-loss} / \epsilon\text{-loss} / v\text{-loss}$ 。
- 在不同设置上测试：
 - 64×64 , patch=4。
 - 256×256 , patch=16。
- 总体结论：
 - 小 patch 场景： $v\text{-pred}$ 仍然很强。
 - 大 patch 场景： $x\text{-pred} + v\text{-loss}$ 显著优于其他组合。

实验细节： 64×64 , patch=4 (小 patch)

- 论文中的 Table 2 (64×64 -p4):
 - 对比不同预测目标和 loss 组合。
- 一个关键观察：
 - v-pred + v-loss 的 FID 最好。
 - x-pred + v-loss 略差一些。
- 说明：
 - 在 token 维度不高、patch 不大的 regime 下，
 - 传统的 v-pred 仍然是非常好的选择。

实验细节：256×256, patch=16（大 patch）

- 另一张关键表：256×256 + p16。
 - patch 维度达到 768（像素空间）。
- 在这个设定下：
 - 几乎所有 v-pred / ϵ -pred 的组合都训练非常困难。
 - 只有 x-pred + v-loss 能得到不错的 FID。
- 这里，“256 分辨率”和“p=16”同时发生变化：
 - 文中没有严格控制变量。
 - 但从后续更多实验可以看出，真正关键的是 patch 维度。

- 1 前置知识：扩散模型与空间选择
- 2 论文结构与主线
- 3 论文核心内容：方法与实验
- 4 理论解释：流形假设与高维 token
- 5 更深入的对比与个人分析
- 6 相关工作与对比
- 7 总结与展望

流形假设（1）：直观理解

- 图像空间维度极高：
 - 例如 $256 \times 256 \times 3 \approx 20$ 万维。
- 但“自然图像”并不在整个空间里均匀分布：
 - 只能出现在一些非常特殊的“形状”上。
- 流形假设：
 - 自然图像集中在一个低维流形上。
 - 你可以想象是一块“弯曲的低维表面”嵌在高维空间里。

流形假设 (2)：噪声 vs 图像

- 图像 x :
 - 在低维流形上。
 - 有大量结构和冗余，比如边缘、纹理、物体。
- 噪声 ϵ :
 - 高斯噪声在高维空间里几乎是“满的”。
 - 没有什么明显的低维结构。
- 结论：
 - 学 $p(x)$ = 学一个低维流形上的分布。
 - 学 $p(\epsilon)$ = 学一个高维、没结构的分布。
 - 后者明显更困难，特别是在高维空间中。

toy 实验：高维空间中预测什么更难？

- 论文中做了一个简单实验：
 - 先在 2D 里生成一个流形上的分布。
 - 再用随机投影把它映射到 D 维（例如 $D = 2, 8, 16, 512$ ）。
- 在这个 D 维空间中：
 - 训练小型扩散模型，分别预测 $x / \epsilon / v$ 。
- 结果：
 - 当 D 较大时，只有 x -pred 还能学好。
 - 预测 ϵ 和 v 的性能迅速崩溃。

Transformer 的 factorization 视角

- 可以把 Transformer 看成在做分布的 factorization:
 - 对每个 token, 用同一套参数建模。
 - attention 只是在 token 之间传播信息。
- 换句话说:
 - 模型对“单个 token”的建模能力受到 hidden 维度限制。
 - token 维度越大, 单个 token 的任务就越难。
- 在大 patch 场景:
 - token 自身就变成高维随机变量。
 - 再去拟合噪声, 基本是“维度灾难”。

高维 patch + hidden 瓶颈

- JiT 的设置：
 - patch 维度：768 / 3072 / 12288。
 - hidden 大约为 768~1800。
- 这意味着：
 - embedding 层必须把高维 patch 强行压缩到 hidden。
 - 内部相当于一个 under-complete 的自编码器。
- 对图像 x ：
 - 虽然观测维度高，但有效流形维度低于 hidden。
 - 有机会被 reasonably 重建。
- 对噪声 ϵ ：
 - 有效维度接近观测维度。
 - hidden 无法承载全部信息 → 学不动。

信息维度 vs 网络宽度：bottleneck_dim 和 hidden_size

- 信息维度：
 - 对于每个 patch, proj1 把 $3p^2$ 维像素映射到 bottleneck_dim。
 - 之后所有变换 (proj2、Transformer、MLP) 都在这个子空间上进行。
 - 因此：单个 patch 最多只保留 bottleneck_dim 维的有效信息。
- 网络宽度：
 - hidden_size (如 768/1024/1280) 决定了每层 Q/K/V、MLP 的宽度。
 - hidden 越大，可表达的非线性组合越多，拟合能力越强。
 - 但不会改变“信息维度上限”这个事实。
- 小结：
 - bottleneck_dim：信息瓶颈，决定能保留多少自由度。
 - hidden_size：模型容量，决定在这块低维流形上能做多复杂的变换。

- 1 前置知识：扩散模型与空间选择
- 2 论文结构与主线
- 3 论文核心内容：方法与实验
- 4 理论解释：流形假设与高维 token
- 5 更深入的对比与个人分析
- 6 相关工作与对比
- 7 总结与展望

从“有效 patch size”看 LDM 与 JiT (1)

- 一个直觉：LDM 里的 VAE 下采样，其实也在做某种“大 patch”。
- 例子：
 - 输入： 256×256 像素图。
 - VAE encoder 下采样因子为 8（常见设置）。
 - 得到 latent： 32×32 。
- 再在 latent 上做 DiT：
 - 假设 patch size 为 2。
 - 对 latent 来说是 $p=2$ 的 patch。

从“有效 patch size”看 LDM 与 JiT (2)

- 从原始像素的视角看：
 - VAE 的 downsample $\times 8$ + DiT 的 patch=2。
 - 综合起来，相当于一个“有效 patch”覆盖了约 16×16 像素。
- 也就是说：
 - LDM + DiT 本质上也在用“大 patch”压缩。
 - 只是这一步由 VAE + DiT 一起完成，而不是单一的 patchify 线性层。
- 对比 JiT：
 - JiT 直接在 pixel space 上用 patch=16 的线性 patchify。
 - 不引入单独的 VAE 阶段。

VAE 压缩 vs 线性 patchify: 谁更“聪明”?

- VAE encoder:
 - 多层卷积 + downsample。
 - 每个 latent 单元的感受野覆盖较大的区域。
 - patch 边界两侧在 encoder 里已经互相“看到”对方。
- 线性 patchify:
 - 每个 $p \times p$ patch 被独立展平。
 - 边界两侧信息只在后续 Transformer 中才交流。
- 直觉:
 - 同样是“ 16×16 ”范围的信息，VAE 的压缩更平滑、信息利用更好。
 - patchify 的压缩更粗暴。

VAE 通道数 vs patchify 通道数

- 假设从 256×256 下采样到 32×32 :
 - 每个 latent cell 覆盖原始约 8×8 的区域。
- 对比通道数：
 - 线性 patchify $p=8$: patch 维度 = $8 \times 8 \times 3 = 192$ 。
 - VAE latent: channel 数往往远小于 192 (比如 4 或 8)。
- 说明：
 - VAE 的 encoder 是一个“强大的非线性压缩器”，把信息挤到更低维的 latent。
 - patchify 则是“高维线性展开”，让模型面对的 token 分布非常高维。

LDM 的优点与痛点

- 优点:
 - 通过 VAE 大幅降低维度, 扩散训练效率高。
 - latent 空间更规整 (KL 正则), 方便预测噪声。
- 痛点:
 - latent 尺度是一个信息瓶颈, 不能无限放大。
 - VAE 和扩散是两阶段训练, 目标不完全一致。
 - 如果 latent 太大, 扩散又会变得难学。
- 这使得:
 - latent-space 与 pixel-space 的取舍变成一个核心设计问题。

JiT 的位置：另一种“大 patch + 压缩再生成”

- 从宏观视角看：
 - 几乎所有图像生成方法都在做“压缩再生成”：
 - 先用某种方式压缩空间维度（大 patch / VAE 等）。
 - 再在压缩后的表示上建模。
- LDM：
 - 压缩部分由 VAE 完成。
 - 再在 latent 上做扩散。
- JiT：
 - 压缩部分由 patchify + embedding 完成。
 - 直接在 pixel-space 的大 patch token 上做扩散。

JiT 的优势：端到端、不依赖独立 VAE

- JiT 的特点：
 - 不需要单独训练 VAE。
 - 整个模型从噪声到图像是一条端到端链路。
- 相比 LDM：
 - 没有“两阶段目标不一致”的问题。
 - 没有单独的重构损失 / perceptual loss / GAN loss。
- 局限：
 - 当前 patchify / unpatchify 太简单，压缩质量还不如 VAE。
 - 端到端训练难度高，对配方要求更严格。

一个设想：JiT 能否最终平替 VAE 路线？

- 设想：

- 如果我们能设计出一种“像 VAE 一样强，但可以和扩散一起端到端训练”的压缩/解码结构。
- 再结合 JiT 的 $x\text{-pred} + v\text{-loss}$ 。

- 理论上：

- 当 $t = 1$ 时，JiT 的“去噪”等价于“重建干净图像”。
- VAE 重建可以看作 JiT 的一个特例子任务。

- 实际上：

- 直接照搬 VAE 架构端到端训练，很可能训不动（太复杂）。
- 需要更精心设计的网络和训练策略。

小 patch 场景：v-pred 仍然是最优选

- 论文 Table 2 (64×64 , $p=4$) 显示：
 - v-pred + v-loss 的 FID 最好。
 - x-pred + v-loss 稍差。
- 结合其他实验：
 - 当 patch 维度较小、 $\text{hidden} \geq d_{\text{patch}}$ 时，
 - 传统 v-pred 不存在高维噪声难学的问题，表现很好。
- 说明：
 - JiT 并不是宣称 “x-pred 一统江湖”。
 - 而是指出在 “大 patch、高维 token” 的 regime 下要改用 x-pred。

论文表格的一个小缺陷：变量没有完全控制

- 论文里的两个主表：
 - 一个是 64×64 , $p=4$ 。
 - 一个是 256×256 , $p=16$ 。
- 这意味着：
 - 分辨率从 64 提升到 256。
 - 同时 patch size 从 4 提升到 16。
- 问题：
 - resolution 和 patch size 两个变量一起动。
 - 单看表格，很容易误解为“高分辨率本身需要 x-pred”。

通过额外实验：把“罪魁祸首”锁定到 patch size

- 在固定分辨率（例如 128×128 ）下：
 - 系统测试 $p=1,2,4,8$ 等不同 patch。
 - 分别对比 v-pred 和 x-pred。
- 观察：
 - p 小时：v-pred 最好。
 - p 逐渐增大：v-pred 质量下降，x-pred 优势显现。
- 结论：
 - 真正触发“必须用 x-pred”的，是 patch 维度变大。
 - 与是否像素空间、分辨率多大关系不大。

patchify 的问题：通道太多，分布太难

- 从“能力”角度看：
 - patchify 本身不是弱算子。
 - 线性/卷积层可以表达任意线性压缩。
- 真正的问题在于：
 - 输出维度 d_{patch} 太大（例如 768、3072）。
 - 模型要拟合这个高维空间里的噪声分布本身就很难。
- 对比 VAE：
 - VAE 把一个大块区域的信息压到比 768 小很多的 latent。
 - patchify 基本是直接把所有像素拼起来。

新一代 pixel DiT：更聪明的解码方式

- 最近的一些 pixel DiT 工作：
 - 不再直接让每个 token 输出 $p \times p \times 3$ 个像素。
 - 而是：
 - 用更复杂的 decoder（例如像素 Transformer）。
 - 让 decoder 只看局部像素 + 之前的 patch 特征。
- 目的：
 - 减轻单个 token 的高维输出负担。
 - 让 pixel-space 生成更加高效、可训练。
- JiT 正好处在这个方向的早期版本：
 - 非常简洁的 patchify/decoder。
 - 强调预测目标选择带来的影响。

- 1 前置知识：扩散模型与空间选择
- 2 论文结构与主线
- 3 论文核心内容：方法与实验
- 4 理论解释：流形假设与高维 token
- 5 更深入的对比与个人分析
- 6 相关工作与对比**
- 7 总结与展望

Simple Diffusion / SiD2: 基本设定

- Simple Diffusion / SiD2:
 - 完全在 pixel-space 上做扩散。
 - 不使用 VAE，不使用 latent token，不做 patchify。
 - 采用 U-ViT (U-Net 形状的 Vision Transformer) 结构。
- 主要关注点：
 - 设计更合理的噪声 schedule 和 SNR 曲线。
 - 使用接近 flow-matching 的时间参数化。
 - 让 pixel-space 的训练更稳定。
- 结论：
 - 证明了“像素空间扩散本身”并不是注定训练不动。
 - 合理设计后，pixel-space 也可以达到很强的 FID。

SiD2: 架构与训练特点

- U-ViT 架构:
 - 类似 U-Net, 有多次下采样/上采样。
 - 在低分辨率层使用 self-attention, 在高分辨率层使用更局部的操作。
 - 这样既能处理高分辨率像素, 又避免了超大的 N^2 。
- 训练技巧:
 - 改良的 SNR 调度, 使梯度在高分辨率下不过度衰减。
 - 更合适的 loss (例如 sigmoid loss) 和归一化方式。
- 结果:
 - 可以在 256×256 、 512×512 像素空间上稳定训练。
 - FID 达到当前 pixel-space 中的 SOTA 水平。

SiD2 vs JiT：性能与定位对比

- 性能对比（大致趋势）：
 - ImageNet-256：
 - JiT 最强模型：FID ≈ 2 左右。
 - SiD2：FID $\approx 1.5 \sim 1.6$ ，更好。
 - ImageNet-512：
 - JiT：FID 在 3 左右。
 - SiD2：FID ≈ 1.5 ，是当前最强像素扩散之一。
- 架构与定位：
 - SiD2：
 - 多尺度 U-ViT，结构复杂，训练更贵。
 - 目标是 pixel-space 上的 SOTA 性能。
 - JiT：
 - 纯 ViT，大 patch，结构极简。
 - 目标是理解“大 patch + x-pred”在 pixel-space 下的可训练性。
- 可以这样理解：
 - SiD2：更“工程化”的路线，追求极致 FID。
 - JiT：更“概念 + 理论”的路线，解释大 patch DiT 为什么能训/不能训。

Stable Diffusion 3 (SD3): rectified flow 在大规模系统中

- SD3 的特点：
 - 使用 rectified flow 框架（与 JiT 相同大类）。
 - 使用 logit-normal t 采样策略。
 - 在 latent-space + Transformer 上实现大规模多模态生成。
- 与 JiT 的关系：
 - 在“时间轴/噪声调度”上理念相通。
 - JiT 则更多关注“大 patch 时预测什么”的问题。
- 可以把 JiT 看作：
 - 在 SD3 类似时间配方下，对 pixel-space DiT 的一个特殊改造。

PixelFlow / PixelNerd: pixel DiT 的其他思路

- PixelFlow:
 - 从低分辨率开始，一边去噪一边上采样。
 - 类似“金字塔式”的生成流程。
- PixelNerd:
 - 用 DiT 作为 encoder。
 - 用 NeRF-like 网络作为 decoder 生成像素。
- 它们与 JiT 的区别：
 - 引入了更复杂的解码器结构，以缓解单个 patch 的输出负担。
 - JiT 则保持结构极简，专门研究“预测目标”这一维度。

- 1 前置知识：扩散模型与空间选择
- 2 论文结构与主线
- 3 论文核心内容：方法与实验
- 4 理论解释：流形假设与高维 token
- 5 更深入的对比与个人分析
- 6 相关工作与对比
- 7 总结与展望

JiT 的核心贡献（总结版）

- 指出问题：
 - 大 patch 的 pixel DiT 在传统 v-pred / ϵ -pred 下训练不稳定、质量严重退化。
- 提出方案：
 - 在 rectified flow 框架中使用 x-prediction + v-loss。
- 理论解释：
 - 流形假设 + 高维 token + hidden 瓶颈。
 - 图像在低维流形上，噪声占满高维空间。
- 实验验证：
 - 小 patch: v-pred 仍然最优。
 - 大 patch: x-pred 显著优于 v-pred。

JiT 的定位与局限

- 更精确的定位：
 - JiT 是“大 patch + 高维 token + DiT”场景下的专用配方。
- 不主张：
 - 所有扩散模型都换成 x-pred。
 - 小 patch / latent DiT 场景沿用 v-pred 完全没问题。
- 局限：
 - 仍然需要大量算力。
 - patchify / unpatchify 过于简单，压缩质量不如 VAE/U-ViT。

个人判断：端到端方案的前景

- 从研究味道上看：
 - LDM 代表“强 tokenizer + latent”路线。
 - JiT 代表“端到端 pixel-space DiT”路线。
- 端到端方案的潜力：
 - 不受两阶段训练限制。
 - 理论上可以学到更适配生成任务的表示。
- 难点：
 - 配方更敏感：预测目标、patch size、结构细节都很关键。
 - 需要持续的工程和理论投入。

Takeaways: 对我们自己做研究的启示

- 在设计扩散模型时，必须同时考虑：
 - 空间：pixel vs latent。
 - patch size 和 token 维度。
 - 预测目标： $x / \epsilon / v$ 。
 - 模型容量：hidden 是否成为瓶颈。
- 不要只盯着“数学上哪个目标更优”：
 - 最终要看的是“在给定结构和数据上，哪个更好学”。
- JiT 的价值：
 - 提供了一个清晰的案例：当 token 高维时，预测什么会改变一切。

Q & A

欢迎讨论和提问

8 附录：实现细节与额外实验

9 附录：CFG interval 详解

10 附录：Qualitative Results

附录 A: Implementation Details 总览

- 代码实现：严格跟随 DiT 与 SiT 的公开代码库。
- 配置汇总在 Tab. 9:
 - 模型规模 (B/L/H/G: 深度、hidden 维度、head 数)。
 - 图像大小、patch size、bottleneck 维度。
 - 训练超参: epochs、batch size、学习率、优化器等。
- 附录 A 主要讲 4 件事:
 - ① 时间步 t 的分布 (logit-normal)。
 - ② 高分辨率 (512/1024) 时的 JiT/32 设定。
 - ③ In-context conditioning、dropout、early stopping。
 - ④ CFG scale 与 EMA 的选择, 以及 LLM 风格模块。

附录 A.1: 时间分布 (logit-normal over t)

- 训练时按照 SD3 类似的做法，使用 **logit-normal** 采样 t :

$$s \sim \mathcal{N}(\mu, \sigma^2), \quad t = \text{sigmoid}(s)$$

- 超参数:
 - 在 ImageNet 256/512/1024 上默认 $\mu = -0.8$ 。
 - 方差 $\sigma = 0.8$ 固定。
- 直观理解:
 - 不再让 t 在 $[0, 1]$ 均匀分布。
 - 通过调节 μ ，控制更偏向噪声阶段还是偏向干净图像阶段。
 - 在对图像质量更敏感的阶段采样更密集。

附录 A.2：高分辨率与 JiT/32

- 在 ImageNet 512×512 上采用 JiT/32：
 - patch size = 32，每张图 patch 数 $16 \times 16 = 256$ 。
 - 与 256×256 上的 JiT/16 (patch=16) **token 数完全相同**。
- 差异：
 - 256 分辨率、 $p=16$ ：patch 维度约 $16^2 \times 3 = 768$ 。
 - 512 分辨率、 $p=32$ ：patch 维度约 $32^2 \times 3 = 3072$ 。
- 含义：
 - self-attention 的序列长度 N 不变， N^2 成本类似。
 - 但单个 token 维度显著增大，进一步加剧“大 patch 高维 token”的问题。

附录 A.3: In-context conditioning 与 Dropout

- In-context class tokens:
 - 用一组 class tokens（例如 32 个）注入类别条件。
 - 在某层之前开始插入：B 第 4 层，L 第 8 层，H/G 第 10 层。
 - 配置见 Tab. 9 的 “in-context start block”。
 - 实验表明可显著改善 FID（约 1.2）。
- Dropout 与 early stopping:
 - JiT-H/G 对中间一半的 blocks 使用 dropout（例如 0.2）。
 - 同时作用于 attention 和 MLP。
 - G 规模模型仍易过拟合，需监控 FID 并在恶化时提前停止训练。

附录 A.4: CFG scale 与 EMA 超参

- CFG scale:
 - 推理时在一组候选值（如 1.0–4.0）上搜索最优 CFG。
 - 不把 CFG 固定死，防止对某些模型规模有偏向。
- EMA（指数滑动平均）：
 - 训练期间维护多组不同 decay 的 EMA 权重副本。
 - 可以放在不同 GPU 上，额外成本很小。
 - 推理时选 FID 最优的 EMA 版本。
- 观点：
 - CFG 与 EMA decay 本质上是 **推理阶段的选择**。
 - 不认真调，会让不同方法的比较不公平。

附录 A.5: SwiGLU / RMSNorm / RoPE / qk-norm

- 论文写道：参考 [73]，加入几种通用改进：
 - **SwiGLU**：门控激活，梯度更平滑，训练更稳。
 - **RMSNorm**：只标准化方差，不减均值，比 LayerNorm 更鲁棒。
 - **RoPE**：旋转位置编码，提供相对位置信息，适合长序列/大 patch。
 - **qk-norm**：对 Q, K 做归一化，控制 attention logits 尺度。
- 这些模块最早在大语言模型中被验证有效，JiT 直接移植到 pixel DiT。
- 附录的消融也表明：缺少这些模块时，训练更不稳、FID 更差。

附录 B: Additional Experiments 总览

- B.1: 训练 loss 曲线与单步去噪图 (Fig. 7)。
- B.2: Noise-level shift (Tab. 3)。
- B.3: Bottleneck 线性嵌入 (Fig. 4)。
- B.4: EDM-style pre-conditioner (Tab. 10)。
- B.5: 额外分类损失 (Tab. 11)。
- B.6: 跨分辨率生成 (Tab. 12)。
- B.7: Precision / Recall (Tab. 13)。

附录 B.1: 训练 loss 与单步去噪 (Fig. 7)

- 在同一个 v-loss 空间下比较:

$$L = \mathbb{E} \|v_{\theta}(z_t, t) - v\|_2^2$$

- v-pred: 网络直接输出 v_{θ} ; x-pred: 先预测 x_{θ} , 再转成 $v_{\theta} = (x_{\theta} - z_t)/(1 - t)$ 。
- 结果:
 - 相同 loss 空间下, x-pred 的 loss 明显更低, v-pred 高约 25%。
 - ϵ -pred loss 约高 3 倍, 训练不稳定。
- 单步去噪图:
 - x-pred 的 denoised 图更干净;
 - v-pred 有明显伪影, 这些误差在多步 ODE 中会累计成 FID 差异。

附录 B.2: Noise-level shift (Tab. 3, 设置)

- 场景: JiT-B/16, ImageNet 256×256 , patch=16。
- 问题: 大 patch 下 ϵ/v -pred 的崩溃, 会不会只是噪声调度没调好?
- 做法: 只改 logit-normal 里 t 的均值 μ :

$$s \sim \mathcal{N}(\mu, \sigma^2), \quad t = \text{sigmoid}(s)$$

- $\mu = 0.0, -0.4, -0.8, -1.2$, 对应从“噪声更小”到“噪声更大”。
- 其他设置: 200 epochs, 带 CFG, 评估 FID-50K。

附录 B.2: Noise-level shift (Tab. 3, 结论)

- 对 x-pred:
 - FID: $14.44 \rightarrow 9.79 \rightarrow \mathbf{8.62}$ ($\mu = -0.8$ 最好) $\rightarrow 8.99$ 。
 - 合理提高噪声水平对 x-pred 有帮助。
- 对 ϵ/v -pred:
 - ϵ -pred 始终在 300 以上的 FID; v-pred 在百级别。
 - 调 μ 只能略微改善，仍然是灾难性失败。
- 结论:
 - noise level 的确需要调，但不足以救回 大 patch 下 ϵ/v -pred 的崩溃。
 - 根本问题在于：高维噪声本身难以建模，预测目标选错了。

附录 B.3: Bottleneck 线性嵌入 (Fig. 4, 设置)

- 仍然是 JiT-B/16, ImageNet 256×256 。
- 原始 patch: 维度为 $16 \times 16 \times 3 = 768$ 。
- 嵌入方式:
 - 两层线性: $768 \rightarrow d' \rightarrow \text{hidden}$ 。
 - d' 为 bottleneck 维度: 16, 32, 64, 128, 256, 512。
- 图中:
 - 横轴: d' (对数刻度)。
 - 纵轴: FID-50K。
 - 黑线: 不使用 bottleneck (直接 $768 \rightarrow \text{hidden}$)。
 - 蓝线: 使用不同 d' 的 bottleneck。

附录 B.3: Bottleneck 线性嵌入 (Fig. 4, 结论)

- 现象：
 - 蓝线整体 低于 黑线，说明加入 bottleneck 整体有利。
 - 最优点在 $d' \approx 64$ ，FID 最低（约 7.3）。32、128 也不错。
 - 即便 $d' = 16$ 或 32，x-pred 模型仍能正常工作，只是 FID 略差。
- 含义：
 - 原始 768 维 patch 是过高维的表示，信息高度冗余。
 - 图像 patch 的有效流形维远小于 768，可以先压缩到几十维再交给 Transformer。
 - 与高维噪声形成对比：噪声的有效维接近观测维，这样强的 bottleneck 会直接毁掉可学性。

附录 B.4: EDM-style pre-conditioner (Tab. 10)

- EDM 中常用形式:

$$x_{\theta}(z_t, t) = c_{\text{skip}} \cdot z_t + c_{\text{out}} \cdot \text{net}_{\theta}(z_t, t)$$

- JiT 结合自己的噪声调度，重新推导这些系数，对比多种 prediction 与系数组合。
- 结果 (ImageNet 256, JiT-B/16):
 - 纯 x -pred ($c_{\text{skip}} = 0, c_{\text{out}} = 1$) FID 最好 (约 10.1)。
 - 引入 EDM-style pre-conditioner 后, $x/\epsilon/v$ 的 FID 均明显变差。
- 结论:
 - 一旦 $c_{\text{skip}} \neq 0$, 输出不再是“纯 x 预测”, 又混入了噪声项。
 - 根据流形假设, 这样不能解决高维噪声难以预测的根本问题。

附录 B.5: 额外分类损失 (Tab. 11)

- 主文强调：不使用 perceptual / GAN / 额外分类损失。
- 附录中做了一个探索：
 - 在中间层挂一个分类头：全局平均池化 + 线性层。
 - 使用 cross-entropy 监督 ImageNet 分类。
 - 分类损失 L_{cls} 乘权重 λ 加到主 loss 上。
 - 为防止 label leakage: 分类层之前的部分不使用 class 条件。
- 结果：
 - JiT-B/16: FID 从 4.37 降到 4.14。
 - JiT-L/16: FID 从 2.79 降到 2.50。
- 作者选择：
 - 虽然有提升，但主实验全部不用该损失，保持“极简扩散配方”的设定。

附录 B.6: 跨分辨率生成 (Tab. 12)

- JiT-G/16@256: 在 256×256 、 $p=16$ 上训练。
- JiT-G/32@512: 在 512×512 、 $p=32$ 上训练，token 数与前者相近。
- 固定训练分辨率，只在生成后做上下采样。
- 结果：
 - FID@256:
 - 直接用 JiT-G/16@256: 1.82。
 - JiT-G/32@512 生成后下采样到 256: 1.84。
 - FID@512:
 - JiT-G/16@256 生成后上采样到 512: 2.45。
 - 直接 JiT-G/32@512: 1.78。
- 解读：
 - 高分辨率模型向下采样几乎不损失质量。
 - 低分辨率模型向上采样会缺失高频细节。

附录 B.7: Precision & Recall (Tab. 13)

- 在 ImageNet 256 上报告：
 - FID、Inception Score。
 - Precision / Recall 指标。
- 对比对象：
 - DiT-XL/2, SiT-XL/2 (latent-space DiT)。
 - RAE 等 DiT+autoencoder 方案。
 - JiT-B/L/H/G/16 (本篇方法)。
- 观察：
 - JiT-H/G 的 FID 在 1.8–1.9, 与主流 latent DiT 接近。
 - IS 较高, 说明语义强度和多样性良好。
 - Precision / Recall 与其他方法存在偏好差异。

8 附录：实现细节与额外实验

9 附录：CFG interval 详解

10 附录：Qualitative Results

CFG interval 的基本概念

- 普通 CFG (classifier-free guidance):

$$f_{\text{cfg}} = f_{\text{uncond}} + w(f_{\text{cond}} - f_{\text{uncond}})$$

在每一步去噪都使用同一个 w 。

- CFG interval 的做法：
 - 将时间轴 $t \in [0, 1]$ 划分为早期和中后期两段。
 - 早期：不使用 CFG ($w = 0$)。
 - 中后期：正常使用 CFG ($w > 0$)。
- 直观：
 - 只在图像已有结构时加引导。
 - 在纯噪声阶段不“瞎用力”，避免数值不稳。

CFG interval 在 JiT 中的作用

- 在 pixel-space + 大 patch 场景：
 - token 维度高，早期高噪声阶段本身就不稳定。
 - 此时 cond / uncond 差值主要是噪声，CFG 会放大无意义的噪声差。
- 采用 CFG interval：
 - 借鉴 LightningDiT / SD3 的配方，前若干步 CFG=0，之后 CFG=常数。
 - 显著提升采样稳定性，减轻爆炸和伪影。
- 可以这样总结：
 - 早期：靠“物理”去噪，把图像从纯噪声拉到粗结构。
 - 中后期：再用 CFG 做语义细化，兼顾细节和稳定。

8 附录：实现细节与额外实验

9 附录：CFG interval 详解

10 附录：Qualitative Results

附录 D: Qualitative Results (Fig. 8–11)

- 附录最后展示大量未经筛选的样本：
 - 使用 JiT-G/16 在 ImageNet 256×256 上生成。
 - 多个类别 id，对应类别名称一一标注。
- 一个细节：
 - 这些样例使用的 CFG=2.2，与 FID 评测时完全一致。
 - 没有为了“好看 demo”临时调更大的 CFG。
- 意义：
 - 定性样本与定量指标对应同一配置，避免 demo 特化。
 - 方便把视觉观感与 FID/IS 对齐起来理解。